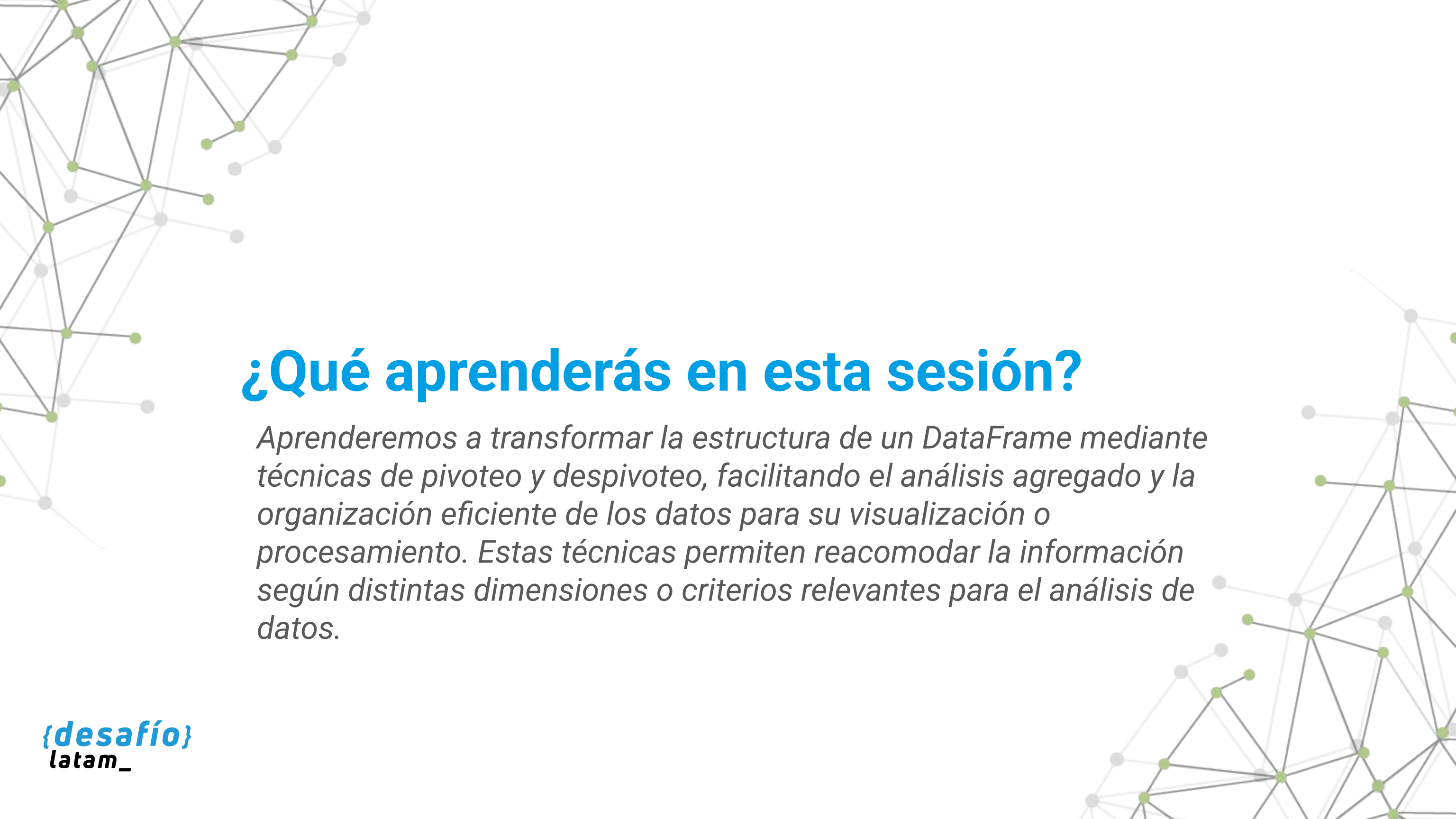




Limpieza y preparación de datos

Clase 6 - Pivoteo y despivoteo de datos en pandas



¿Qué aprenderás en esta sesión?

Aprenderemos a transformar la estructura de un DataFrame mediante técnicas de pivoteo y despivoteo, facilitando el análisis agregado y la organización eficiente de los datos para su visualización o procesamiento. Estas técnicas permiten reacomodar la información según distintas dimensiones o criterios relevantes para el análisis de datos.

Desafío inicial

Imagina que trabajas en el equipo de análisis de una empresa de tecnología educativa. Te han entregado un set de datos que contiene información detallada de participación de estudiantes en distintos cursos durante varios meses. Sin embargo, los datos están en formato largo (una fila por participación). Tu jefatura necesita visualizar rápidamente cuántas participaciones hubo por curso y por mes, en un formato de tabla donde las columnas sean los meses y las filas los cursos.

¿Cómo podrías transformar ese DataFrame para responder a esa solicitud y luego revertir el proceso para obtener nuevamente los registros originales si fuese necesario?

Para resolverlo, aprenderás a usar las funciones `pivot`, `pivot_table` y `melt` de la librería `pandas`, fundamentales para reorganizar, resumir y descomponer estructuras de datos tabulares.

/* Pivoteo de dataframes. */

¿Qué significa pivotear datos en pandas?



Transformación de formato

El pivoteo de datos es una operación que transforma un DataFrame desde un formato largo ("long format") a un formato ancho ("wide format"), donde se reorganiza la disposición de los datos según variables clave. Esto es especialmente útil para crear tablas resumen, generar reportes o preparar datos para visualización.



Proceso de pivoteo

Pivoteo significa transformar un DataFrame de formato largo a uno ancho, en donde:

- Las categorías únicas de una columna se convierten en columnas nuevas.
- Otra columna numérica se distribuye en la tabla como valores.
- Se utiliza un índice como categoría principal que se mantiene como fila.



Beneficios

Este procedimiento permite:

- Organizar datos para reportes de fácil lectura.
- Comparar valores por categoría y dimensión temporal.
- Preparar visualizaciones como mapas de calor o tablas dinámicas.

Pivot con pivot() de pandas

La función pivot() en pandas permite realizar pivoteo siempre que no haya duplicados en la combinación de índice y columnas. Su sintaxis es sencilla, pero tiene limitaciones.

Sintaxis básica

```
DataFrame.pivot(index='...', columns='...',  
values='...')
```

- index: columna que permanecerá como índice de fila.
- columns: valores únicos que se convertirán en columnas.
- values: valores que se ubicarán en la intersección de fila/columna.



Ejemplo práctico de pivot()

```
import pandas as pd
data = {'curso': ['Python', 'Python', 'SQL', 'SQL'],
        'mes': ['Enero', 'Febrero', 'Enero', 'Febrero'],
        'participaciones': [25, 30, 18, 22]}
df = pd.DataFrame(data)
pivotado = df.pivot(index='curso', columns='mes',
                     values='participaciones')
print(pivotado)
```

Resultado:

mes	Enero	Febrero	curso	Python	25	30	SQL	18	22
-----	-------	---------	-------	--------	----	----	-----	----	----

Pandas dataframe transformation



Limitaciones de pivot()



Consideración importante:

Si hay más de un valor para la misma combinación de curso y mes, se generará un error:

```
ValueError: Index contains duplicate entries, cannot reshape
```

Advertencia: pivot() no permite duplicados en la combinación de index y columns. En esos casos, se debe usar pivot_table().

¿Qué pasa si los datos contienen duplicados en las combinaciones de índice y columna? ¿Perdemos la posibilidad de pivotar? Para resolver esto y agregar múltiples registros, utilizamos pivot_table().

Duplicate keys in
pandas dataframe

Pivot con pivot_table(): agregación y mayor control

pivot_table() es una versión más poderosa y flexible de pivot(), ya que permite:

Manejo de duplicados

Trabajar con múltiples valores por combinación.

Funciones de agregación

Agregar con funciones como sum, mean, count, etc.

Agrupación automática

Agrupar automáticamente según reglas estadísticas.

Sintaxis extendida:

```
DataFrame.pivot_table( index='...', columns='...', values='...', aggfunc='...', fill_value=0)
```

Ejemplo de pivot_table()

```
df = pd.DataFrame({
    'curso': ['Python', 'Python', 'SQL', 'SQL',
'Python'],
    'mes': ['Enero', 'Febrero', 'Enero', 'Febrero',
'Enero'],
    'asistentes': [25, 30, 18, 22, 20]
})
pivotado = df.pivot_table(
    index='curso',
    columns='mes',
    values='asistentes',
    aggfunc='sum',
    fill_value=0
)
print(pivotado)
```

Resultado:

```
mes  Enero  Febrero  curso Python 45 30 SQL 18 22
```

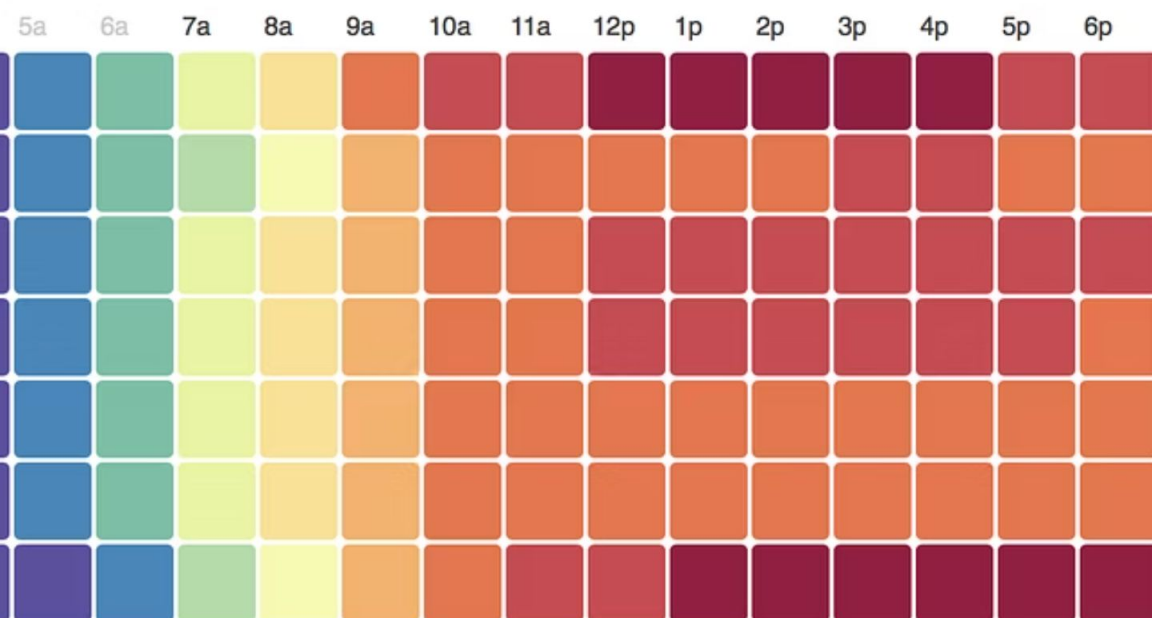


All Day, Every Day

es on Mondays at 9:00 PM. When are you window shopping?



{desafío}
latam_

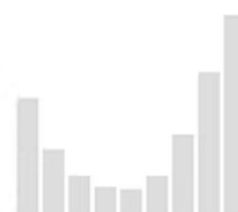


All States

All States



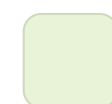
82% Computer
18% Mobile



12a 3a 6a
All traffic daily

nd [Trulia Mobile](#) between June and August 2011. "Computer" traffic includes all house hunting
"mobile" includes app and mobile web traffic from cell phones and tablets. Times are normalized

Buenas prácticas con `pivot_table()`



Manejo de valores nulos

Usar `fill_value` para evitar valores nulos.



Selección de función de agregación

Elegir `aggfunc` adecuadamente según la naturaleza de los datos.



Inclusión de totales

Usar `margins=True` para añadir totales por fila/columna.

Ahora que sabemos cómo reorganizar los datos para visualizarlos mejor, surge la pregunta inversa: ¿qué ocurre cuando queremos volver del formato ancho al formato largo? Aquí es donde entra el despivotado.

/* Despivoteo de dataframes. */

Despivotado con melt(): del formato ancho al largo

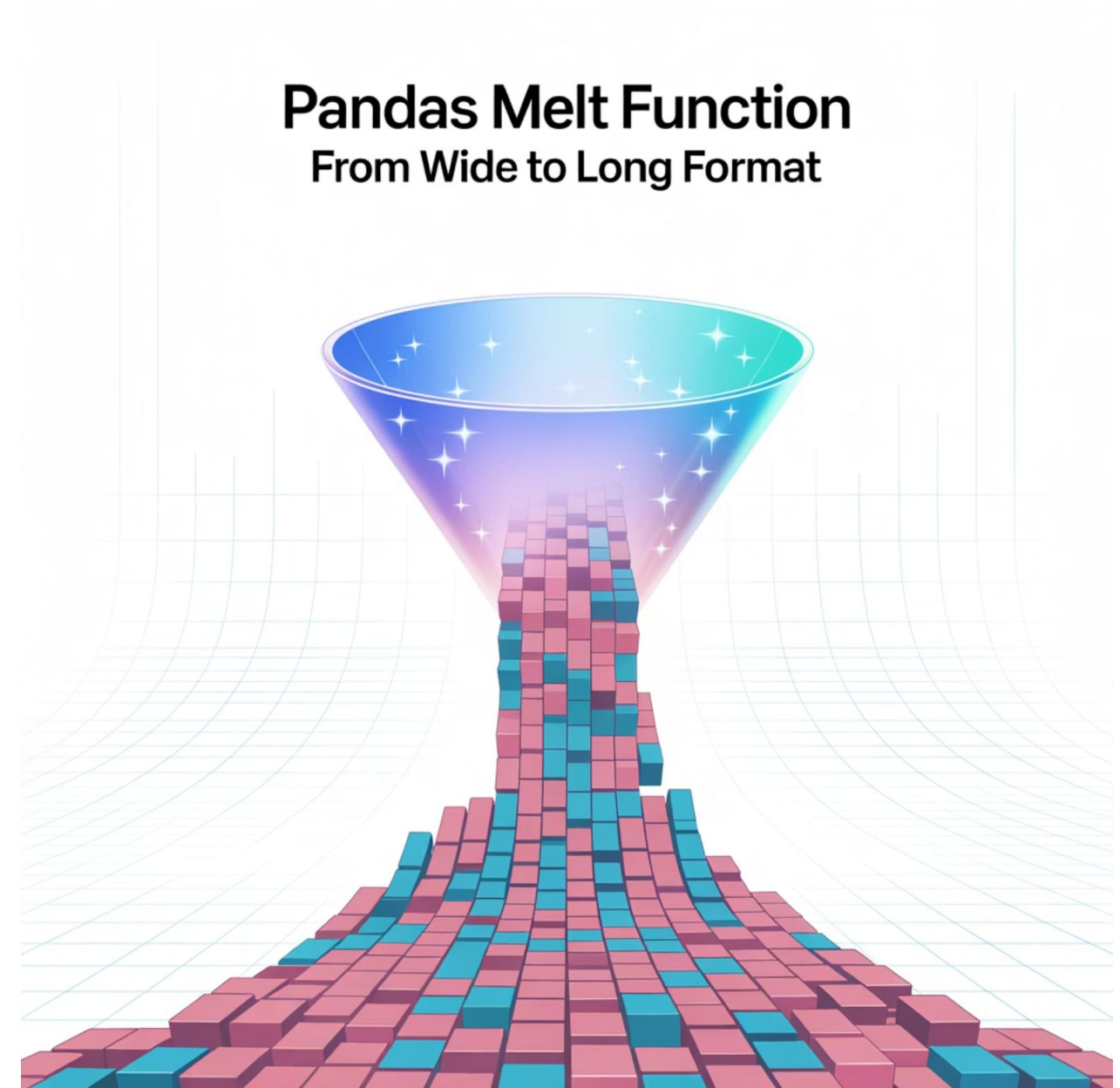
melt() es la herramienta de pandas para revertir estructuras pivotadas.

Transforma las columnas en filas, permitiendo volver a un formato relacional clásico.

Sintaxis

```
pd.melt(df, id_vars=[...], var_name='...', value_name='...')
```

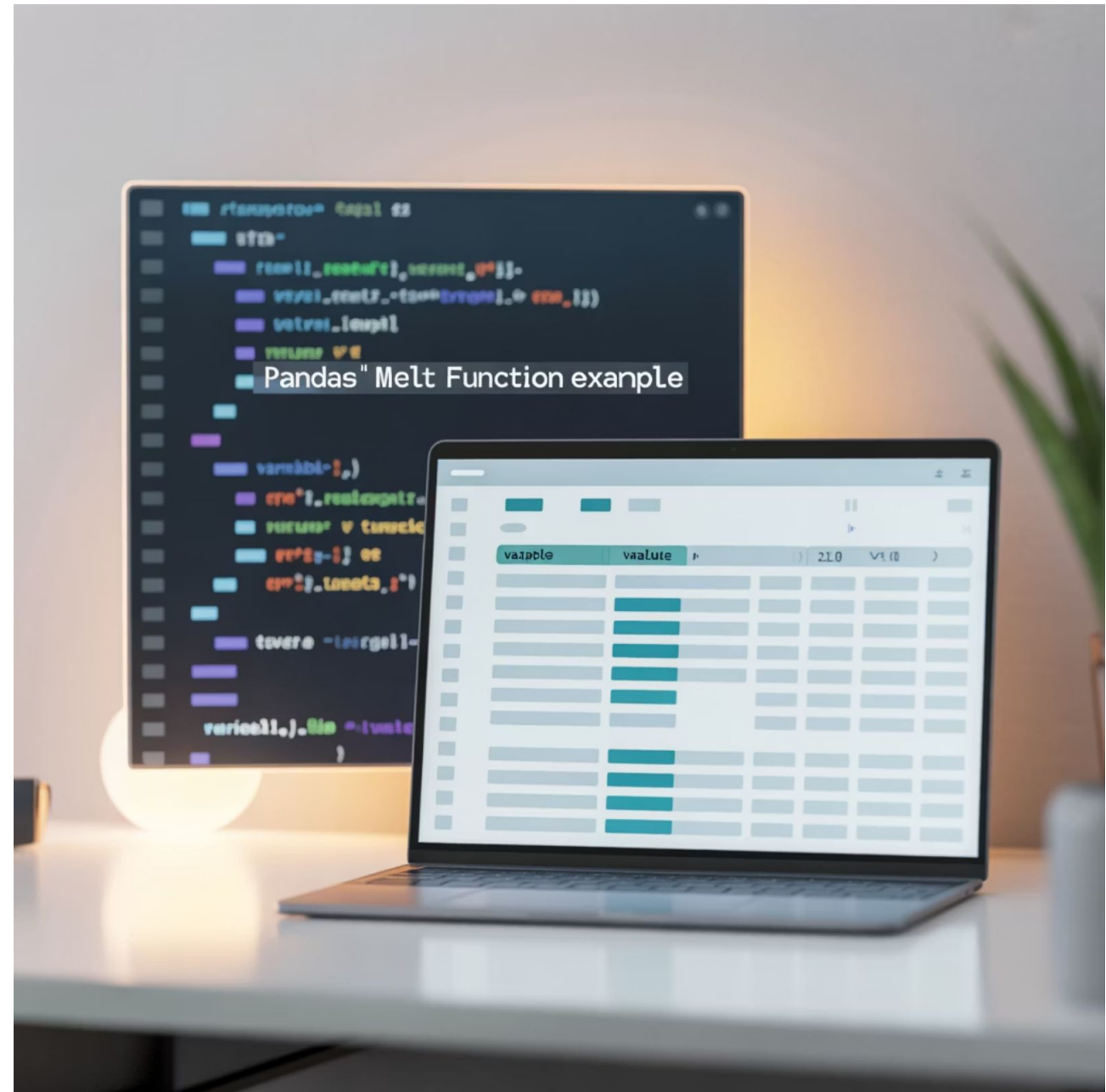
- id_vars: columna(s) que permanecerán fijas.
- var_name: nombre que se asignará a las columnas convertidas en fila.
- value_name: nombre de la columna que contiene los valores numéricos.



Ejemplo de melt()

```
pivotado = pivotado.reset_index()
despivotado = pd.melt(
    pivotado,
    id_vars='curso',
    var_name='mes',
    value_name='asistentes'
)
print(despivotado)
```

Esto elimina entradas donde el cliente_id se repite, manteniendo la primera.



Actividad guiada: Transformación estructural de asistencias a talleres



Ejercicio

Transformación estructural de asistencias a talleres

Objetivo de la actividad

Aplicar las funciones `pivot_table()` y `melt()` para transformar la estructura de un conjunto de datos, generando un resumen por categorías y luego revirtiéndolo al formato largo para su integración posterior.

Situación problema

Estás trabajando con un conjunto de datos que registra la cantidad de personas que asistieron a distintos talleres durante dos meses. El equipo directivo necesita ver estos datos organizados por taller (filas) y mes (columnas), y posteriormente enviar el dataset a otra unidad que solo acepta datos en formato largo.



Ejercicio

Instrucciones

Cargar el archivo:

```
import pandas as pd
df = pd.read_csv('asistencias_talleres.csv')
```

Pivotear los datos:

```
tabla = df.pivot_table(
    index='taller',
    columns='mes',
    values='asistentes',
    aggfunc='sum'
)
```



Ejercicio

Instrucciones

Despivotear los datos:

```
tabla = tabla.reset_index()
df_original = pd.melt(tabla,
                      id_vars='taller',
                      var_name='mes',
                      value_name='asistentes')
```

Guardar resultados:

```
tabla.to_csv('asistencias_pivot.csv',
             index=False)

df_original.to_csv('asistencias_revertidas.csv',
                  index=False)
```



Preguntas de reflexión y cierre

1 Formato ancho

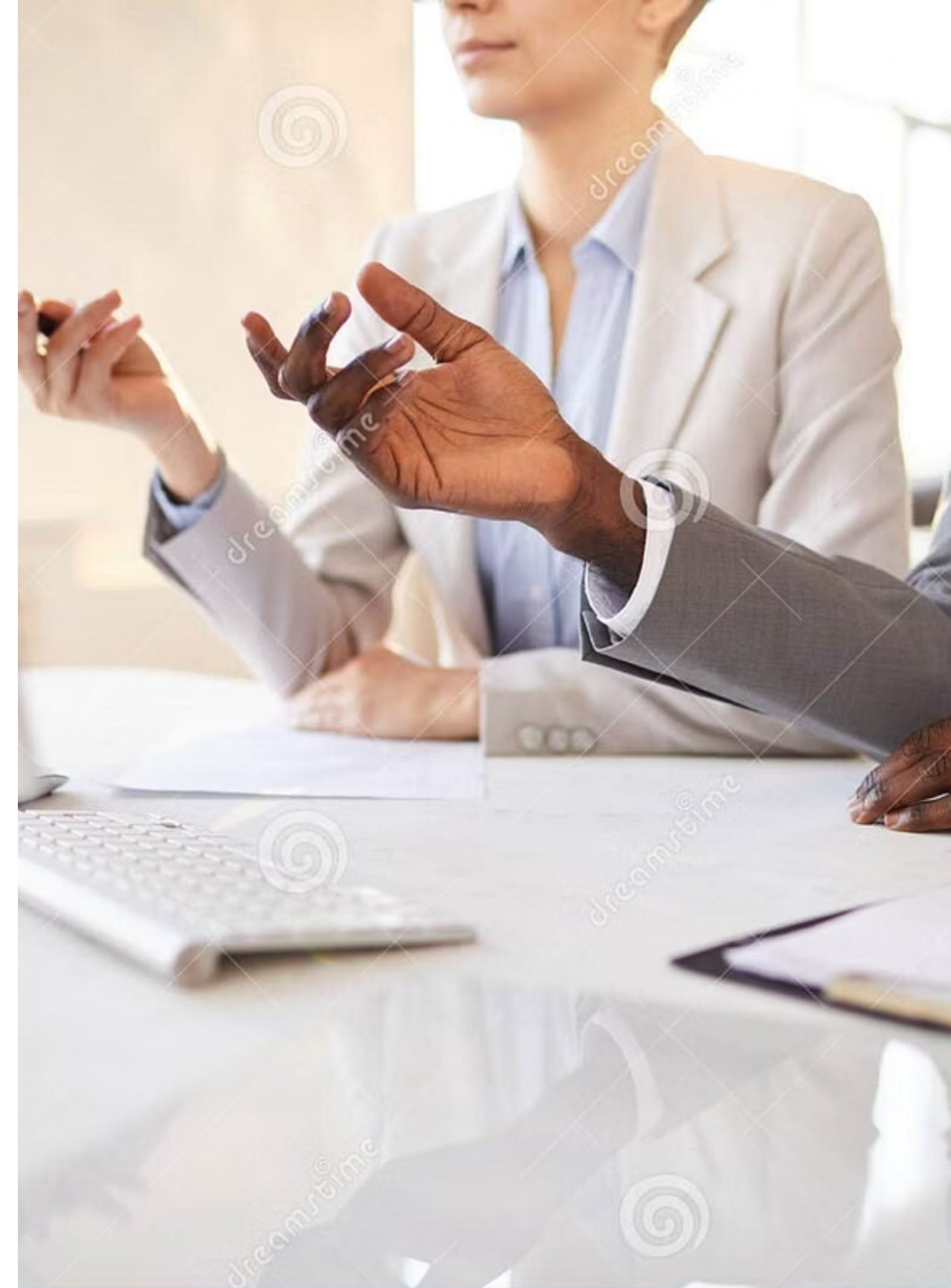
¿En qué casos es mejor usar formato ancho?

2 Riesgos del despivoteo

¿Qué riesgos hay al despivotear sin revisar los datos originales?

3 Aplicación práctica

¿Cómo aplicarías este conocimiento en tu entorno de trabajo?



Actividad práctica autónoma: Resumen mensual de ventas por sucursal



Ejercicio

Transformación estructural de asistencias a talleres

Objetivo de la actividad

Aplicar `pivot_table()` para generar una tabla de resumen de ventas por sucursal y por mes, y luego revertirla usando `melt()` para reenviar el dataset a una unidad que lo requiere en formato largo.

Contexto

Has recibido el archivo `ventas_mensuales.csv`, que contiene datos de ventas por sucursal y mes. La gerencia te pide transformar esta información para mostrar sucursales como filas y meses como columnas, y luego volver al formato largo para almacenamiento.



Ejercicio

Instrucciones

Cargar el archivo:

```
import pandas as pd
df =
pd.read_csv('ventas_mensuales.
csv')
```

Aplicar pivot_table():

```
resumen = df.pivot_table(
    index='sucursal',
    columns='mes',
    values='ventas',
    aggfunc='sum'
)
```

Despivotear con melt():

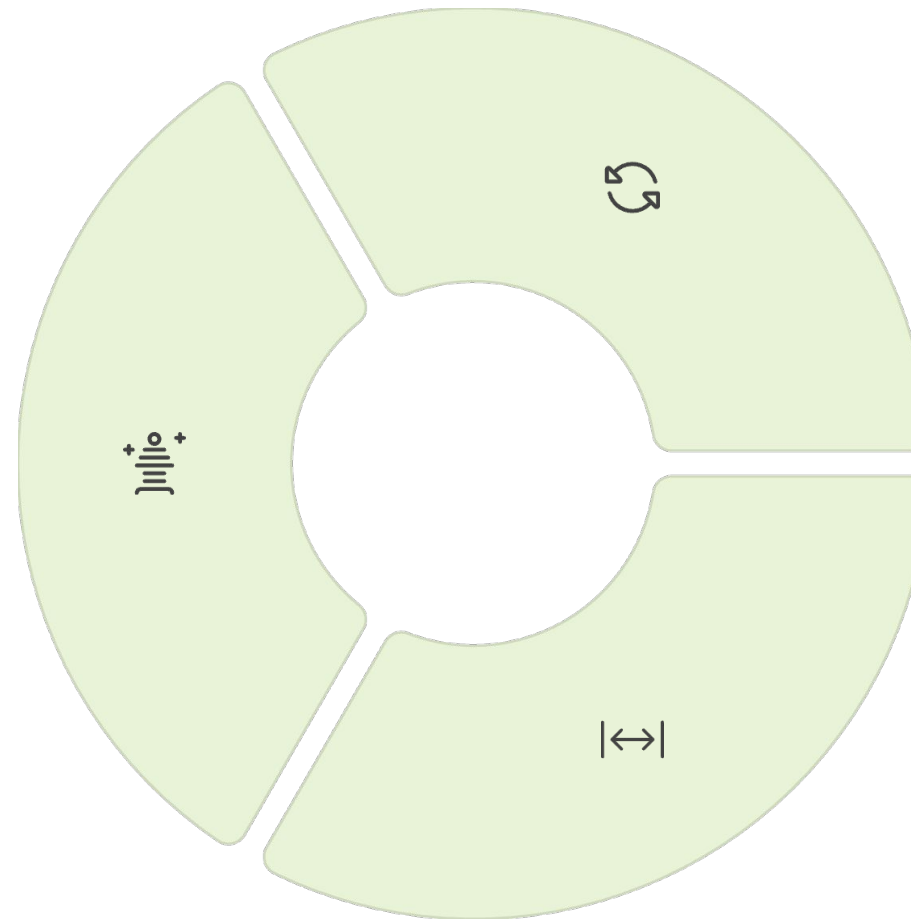
```
resumen = resumen.reset_index()
df_largo = pd.melt(resumen,
                    id_vars='sucursal',
                    var_name='mes',
                    value_name='ventas')
```

Guardar los resultados como archivos .csv.

Resumen

Transformación

Aprendimos a transformar la estructura de datos usando `pivot()` y `pivot_table()`.



Reversión

Utilizamos `melt()` para revertir la estructura de un DataFrame.

Aplicación

Identificamos casos reales en los que se requiere reorganizar datos para facilitar reportes o análisis.



Próxima sesión...

En la siguiente sesión aprenderemos a combinar múltiples DataFrames utilizando técnicas de concatenación y merge.

Estas habilidades nos permitirán unificar información desde diferentes fuentes, integrar datos relacionados por llaves comunes, y preparar datasets más ricos y completos para el análisis exploratorio o la modelación.